Practical - 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Yes. session_start() is called at the very top of process_login.php. On logout, logout.php does three things in order: clears $_SESSION = array(), calls session_destroy(), and deletes the remember-me cookie — this is the correct and secure way to fully terminate a session.

2. Is password_verify() used correctly?

Yes. After fetching the user's row from the database, the script calls password_verify($password, $user['password_hash']). The raw typed password is passed as the first argument and the stored hash from MySQL as the second — exactly the correct usage.

3. Is unauthorized dashboard access blocked?

Yes. check_session.php checks for $_SESSION['student_id']. If it's not set, the user is redirected to login.html?msg=pleaselogin immediately with exit(). Any page that includes this file becomes protected.

4. Is session timeout implemented?

Yes. At the top of process_login.php, if $_SESSION['last_active'] exists and is more than 900 seconds (15 minutes) old, the session is destroyed and the user is sent to login.html?msg=timeout. The last_active value is refreshed on every successful request.

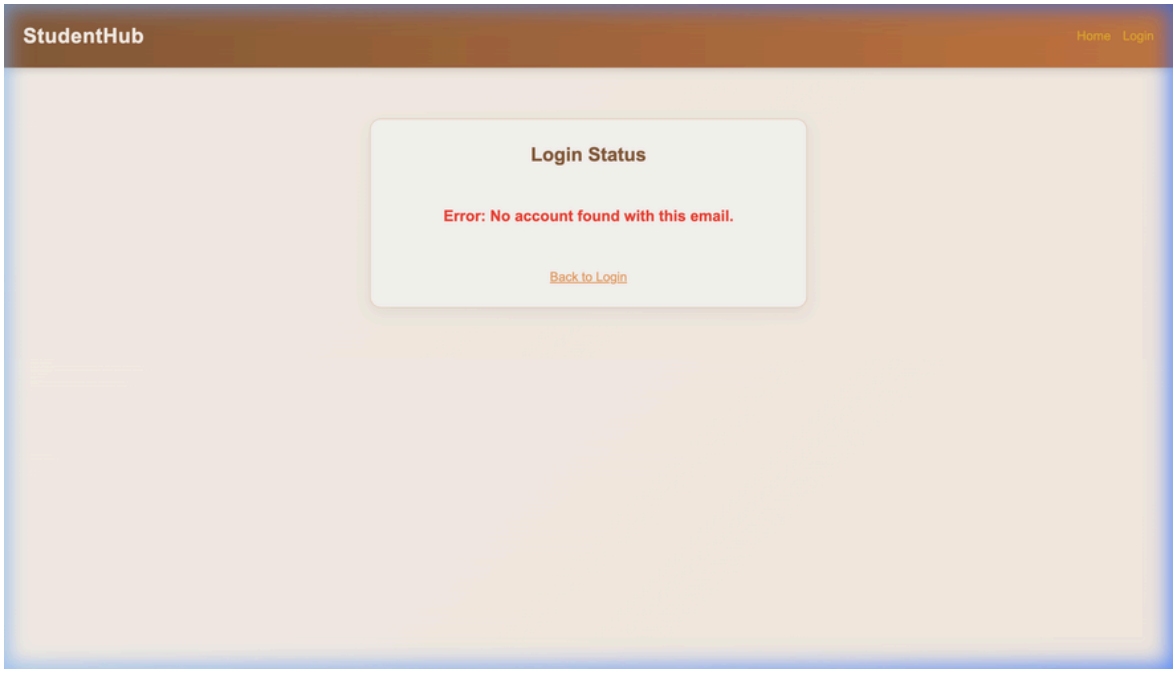
5. Are admin and student users redirected to appropriate dashboards?

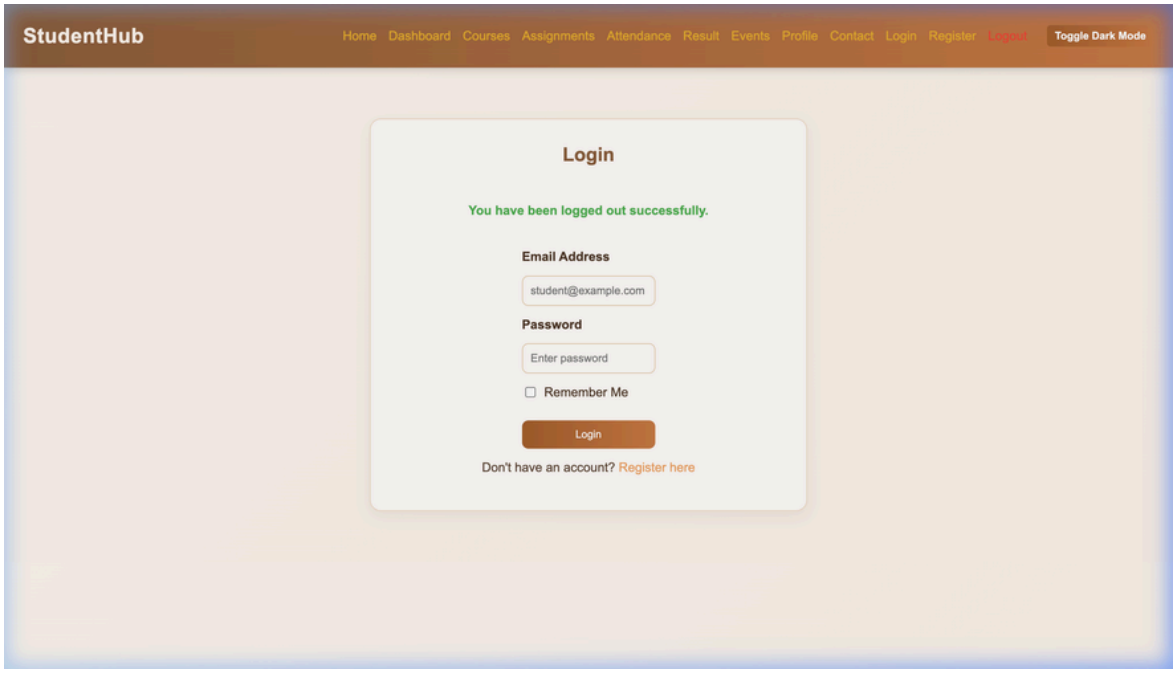
Yes. After a successful login, the script checks $user['role']:
If admin → redirects to dashboard.html?welcome=admin
If student → redirects to dashboard.html?welcome=student

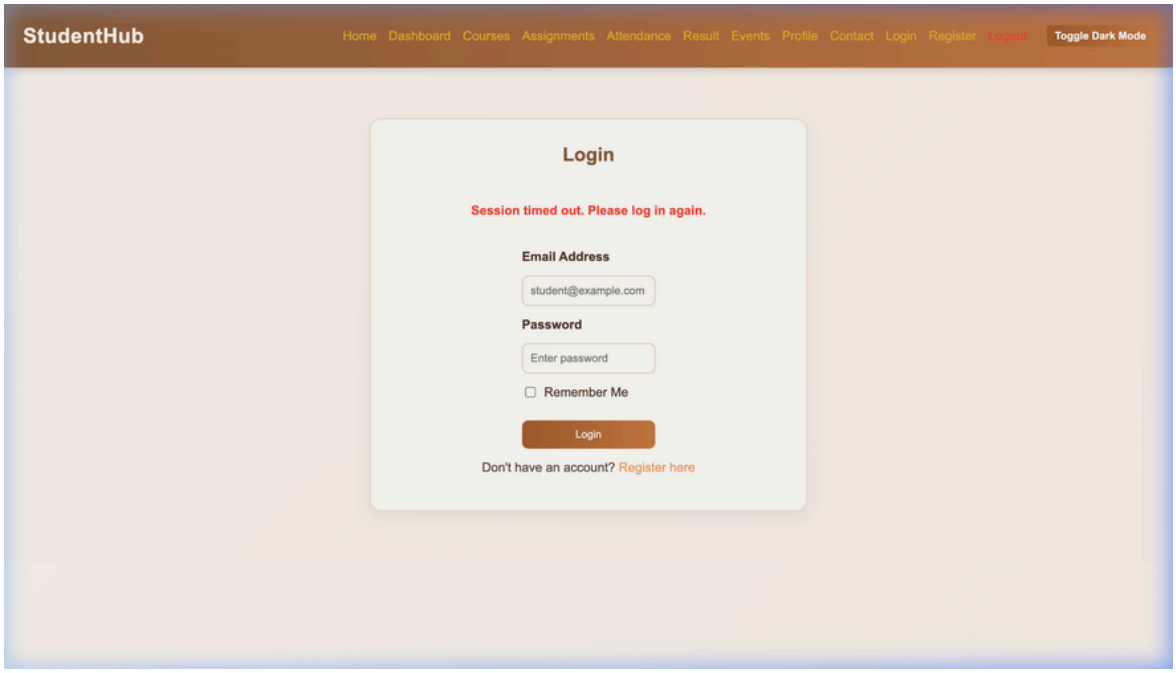
B. Implementation

File	What It Does
login.html	Frontend login form with Email, Password, Remember Me checkbox
process_login.php	Verifies credentials with password_verify(), sets session, handles role-based redirect
logout.php	Clears session, deletes cookie, redirects with success message
check_session.php	Reusable session guard — protects any page that includes it
DATA/update_schema.sql	Adds role column to students table for admin/student distinction

C. Evidence of Work (Screenshots)




StudentHub
Home Dashboard Courses Assignments Attendance Result Events Profile Contact Login Register Logout
Toggle Dark Mode
Login
You have been logged out successfully.
Email Address
student@example.com
Password
Enter password
Remember Me
Login
Don't have an account? Register here


StudentHub
Home Dashboard Courses Assignments Attendance Result Events Profile Contact Login Register Logout
Toggle Dark Mode
Login
Session timed out. Please log in again.
Email Address
student@example.com
Password
Enter password
Remember Me
Login
Don't have an account? Register here

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%210

E. Challenges Faced:

1. Session ID Regeneration — Understanding why session_regenerate_id(true) must be called after verifying login but before storing session variables, to prevent session fixation attacks.
2. Remember Me Cookie — Setting the cookie expiry correctly (time() + 7 * 24 * 60 * 60) and deleting it on logout by setting expiry to a past time (time() - 3600).
3. Password Not Escaped — mysqli_real_escape_string() must NOT be applied to the password before passing it to password_verify(). Escaping the password first would corrupt the comparison.

F. Learning Outcomes:

Learned how PHP sessions work — starting, storing variables, and destroying them.
Understood password_verify() and why it pairs with password_hash() for secure authentication.
Learned session timeout logic using time() and last_active tracking.
Understood role-based redirection — different users get sent to different pages based on their role in the database.
Learned the Remember Me pattern using browser cookies (setcookie()).

Faculty Remarks: